Evaluation Results Package (Deliverable D3)

SBOM Supply Chain Risk Scoring: A Contextual Component Risk Ranking Framework

William A. Davis
CS 690 Graduate Research Project, Fall 2026
Advisor: Dr. Omar Ochoa
Covers Task I (evaluation and sensitivity analysis)
November 7, 2026

---

1. Summary

This package reports the evaluation defined in Section 8 of the D1 Scoring Specification, run on all four test projects with one analysis date, November 6, 2026. The weights were fixed in D1 before any results were seen and were not changed afterwards. Of the three targets set in advance, two were met and one was met only in part:

Target (D1 Section 8)	Result	Met?
Differentiation: at least 3 of the contextual top 10 differ from the CVSS-only top 10, in at least 3 of the 4 projects	Juice Shop 5, Gitea 2, PyGoat 1; WebGoat has only 8 vulnerable components, so a top-10 comparison does not apply	Partly (1 of 3 eligible projects)
Justification: every component that enters the top 10 has exploit probability, exploit evidence or depth among its two largest factors	8 of 8 entering components justified	Yes
Stability: under ±0.05 and ±0.10 weight changes, top-10 overlap ≥ 8 and Kendall's τ ≥ 0.80 in at least 90% of runs, per project	75 of 80 runs stable; every project at 18 of 20 or better	Yes

The contextual ranking reorders the baseline substantially in every project (Kendall's τ between 0.39 and 0.68), and every reordering that reaches the top 10 has a stated cause. What the evaluation does not show is that the reordering usually changes which components make the top 10: in three of the four projects the same high-severity components dominate both lists and the model mainly changes their order. Section 3 discusses why.

2. Data and Method

Each project's SBOM (Task C) was enriched on Will Davis's workstation on November 6, 2026 with OSV [1], NVD [2], FIRST EPSS [3] and the CISA KEV catalog [4], then scored with the D1 weights (severity 0.30, EPSS 0.25, exploit evidence 0.20, depth 0.15, age 0.10). The command sbomrisk evaluate computed every measure below; the files it wrote (evaluation.json, summary.csv, sensitivity.csv and density.csv) accompany this package, and re-running it on the same enrichment files reproduces them exactly.

Project	Ecosystem	Components	Vulnerable	Vulnerabilities	With CVSS and EPSS	In KEV
Juice Shop	npm	645	34	93	90 (97%)	0
Gitea	Go	231	26	105	99 (94%)	1
PyGoat	PyPI	33	14	145	136 (94%)	1
WebGoat	Maven	190	8	56	56 (100%)	1

Kendall's τ [5] compares two complete rankings of the same items; 1.0 means identical order and 0 means no relationship. It is computed over all vulnerable components of a project, not just the top 10. For the sensitivity

analysis each of the five weights was moved by −0.10, −0.05, +0.05 and +0.10 in turn, with the other four rescaled proportionally so the weights still sum to 1, giving 20 runs per project. No component in any project was left unassessed.

3. Differentiation

Project	Ranked	New in top 10	Rank changes	Kendall's τ	Target met
Juice Shop	34	5	31 of 34	0.39	Yes
Gitea	26	2	24 of 26	0.68	No
PyGoat	14	1	13 of 14	0.58	No
WebGoat	8	n/a	7 of 8	0.64	n/a

Juice Shop is the clearest case. Five of its contextual top 10 are outside the baseline top 10, and τ of 0.39 means the two rankings agree only loosely. notevil 1.3.3 moves from #27 to #4: its CVSS score is only 6.5, but it is a direct dependency and NVD lists public exploit code for its vulnerability. uuid, minimatch and ws enter because NVD lists public exploit code for their driving vulnerabilities, and sanitize-html enters as a direct dependency.

The other projects fall short for different reasons. WebGoat has only eight vulnerable components, so every one of them is in both top-10 lists by construction; D1 anticipated this and reports τ alone for such projects (0.64). PyGoat has 14, so at most four components can enter its top 10, and only idna does. In Gitea the order changes a great deal (24 of 26 components move, and andybalholm/brotli rises from #4 to #1 on an EPSS probability of 0.83) but the components near the top are high on every factor, so only golang-jwt and protobuf enter.

Two conclusions follow, and both will be stated plainly in the final report. First, with WebGoat ineligible, the target required success in all three remaining projects, which is a stricter test than the "3 of 4" wording suggests. Second, and more substantively, in small dependency sets the components with the highest CVSS scores also tend to be direct dependencies with old versions, so the factors agree and the top 10 is stable. The model's effect there is on order within the list, which τ captures and the top-10 count does not. The weights were not retuned to meet the target, because doing so after seeing the results would invalidate the evaluation.

4. Justification of Entering Components

Every component that entered a top 10 has a non-severity factor among its two largest contributions. Severity is always the other, which is expected: a component cannot reach the top 10 on context alone, and the model is designed so that it should not.

Project	Component	Baseline rank	Contextual rank	Two largest factors
Juice Shop	notevil@1.3.3	#27	#4	severity, depth
Juice Shop	sanitize-html@1.4.2	#14	#5	severity, depth
Juice Shop	uuid@8.3.2	#23	#8	severity, exploit evidence
Juice Shop	minimatch@3.0.5	#15	#9	severity, exploit evidence
Juice Shop	ws@7.4.6	#19	#10	severity, exploit evidence
Gitea	github.com/golang-jwt/jwt/v5@v5.0.0	#14	#9	severity, depth
Gitea	google.golang.org/protobuf@v1.31.0	#16	#10	severity, depth
PyGoat	idna@3.4	#11	#6	severity, exploit evidence

Four entries are explained by exploit evidence and four by depth. None is explained by EPSS. That is not because EPSS has little effect: it moves components that were already in the baseline top 10, such as Pillow in PyGoat and brotli and x/crypto in Gitea, to the top of the list. High EPSS values in these projects belong to components that also have high CVSS scores.

5. Weight Stability

Project	Stable runs	Lowest top-10 overlap	Lowest τ
Juice Shop	18 of 20	7	0.78
Gitea	20 of 20	9	0.85
PyGoat	19 of 20	7	0.76
WebGoat	18 of 20	8	0.64

75 of 80 runs (94%) met both thresholds, and each project individually met the 90% target. The 5 runs that did not are listed below. All five come from the largest weight changes (±0.10), and none comes from the EPSS weight:

Project	Weight changed	Change	Top-10 overlap	τ
Juice Shop	exploit evidence	−0.10	7	0.86
Juice Shop	age	−0.10	7	0.78
PyGoat	depth	+0.10	7	0.76
WebGoat	depth	−0.10	8	0.64
WebGoat	depth	+0.10	8	0.79

WebGoat's two unstable runs reflect its size more than its weights: with eight components, two adjacent swaps lower τ to 0.64. In Juice Shop, both unstable runs replace the same three components, uuid, minimatch and ws, which entered the top 10 on exploit evidence. Lowering the exploit-evidence weight to 0.10 removes them directly. Removing the age weight (0.10 to 0.00) lets three newer packages (tar, multer and hbs) displace them. Juice Shop's top 10 is therefore the most sensitive to the weights, because its entries from #8 down are separated by small margins. The ranking is otherwise robust: Gitea stayed stable in all 20 runs, and the top-10 overlap never fell below 7.

6. Vulnerability Density by Depth

The table compares the vulnerable components at each depth with all components at that depth, as counted in the Task F dependency tables.

Project	Depth 1	Depth 2	Depth 3	Depth 4 and deeper	Share of total risk score from direct dependencies
Juice Shop	11 of 63 (17%)	15 of 178 (8%)	5 of 179 (3%)	3 of 269 (1%)	36%
Gitea	19 of 110 (17%)	7 of 120 (6%)	0 of 1 (0%)	n/a	74%
PyGoat	9 of 16 (56%)	4 of 14 (29%)	1 of 3 (33%)	n/a	70%
WebGoat	4 of 37 (11%)	2 of 38 (5%)	1 of 38 (3%)	1 of 78 (1%)	61%

In all four projects a direct dependency is far more likely to carry a known vulnerability than a deep one. In Juice Shop, 17% of direct dependencies are vulnerable but fewer than 2% of those four or more levels down. Decan et al. [6] showed that npm projects are commonly exposed to vulnerabilities through transitive dependencies; these results add that, per component, the exposure is much thinner at depth than at the surface. It also supports the depth factor: direct dependencies are where most known vulnerabilities are and where a fix is in the project's own hands. Juice Shop is the exception in risk share (36%) because its most dangerous components, such as jsonwebtoken 0.1.0 (through express-jwt) and lodash 2.4.2 (through sanitize-html), are pulled in transitively by old direct dependencies.

7. Data Quality and Edge Cases

Project	Flag	Count	Effect on score
PyGoat	No CVE alias (GitHub-only advisories)	9	EPSS and KEV not applicable; E = 0
PyGoat	No severity data	1	S = 0.50
Gitea	No CVE alias	6	E = 0
Gitea	No severity data	3	S = 0.50
Juice Shop	No CVE alias	2	E = 0
Juice Shop	CVE too new for an EPSS score	1	E = 0
WebGoat	Release date not found	6	A = 0.50

Every gap is flagged in the output rather than filled silently, and none left a component unranked. Three issues found during evaluation deserve particular mention.

WebGoat release dates. All six missing dates belong to 2026 releases (Guava 33.7.1, both Jackson lines and Tomcat 11.0.24). The tool looks dates up in the Maven Central search index [7], which did not return these recent versions; the two older components it did find were dated correctly. The Last-Modified date of each artifact on repo1.maven.org supplies the missing values (July 3 to August 18, 2026) and matches the index exactly for the two found components. Re-ranking WebGoat with those dates swaps commons-lang3 and Tomcat at #4 and #5, lowers τ from 0.64 to 0.50 and changes stability from 18 to 19 of 20 runs. No conclusion in this package changes.

Upstream over-reporting. OSV occasionally lists a version that the advisory text excludes. CVE-2026-69249 is reported for cryptography 39.0.1 although NVD describes versions 42.0.0 through 48.0.0, and PyPI's own data lists it too, so the error is in the shared upstream record. Likewise messageformat 2.3.0 is matched to GHSA-xfqm-j7pc-xrfc, whose npm range is "below 2.3.0". The tool reports what OSV returns and does not override it.

Data drift. PyGoat was enriched on October 23 for milestone M2 and again on November 6. In those two weeks two new advisories were published and NVD raised one Pillow score from 9.2 to 9.8, which moved Pillow from #4 to #1 in the baseline. Every result therefore records its analysis date, and every API response is cached so that a run can be repeated exactly.

8. Cross-Check Against Established Scanners

To check that the enrichment finds the vulnerabilities established tools find, the matches were compared with the data sources behind pip-audit [8] and npm audit [9]: PyPI's per-version vulnerability records for PyGoat, and the npm registry's bulk advisory service for Juice Shop. Both checks ran on November 7, 2026.

Project	Reference source	Reference finds	Also found by sbomrisk	Found only by sbomrisk
PyGoat	PyPI (pip-audit)	141	141 (100%)	4 (Pillow 2, oauthlib 2)
Juice Shop	npm registry (npm audit)	77	77 (100%)	12 (multer 11, messageformat 1)

Nothing either reference reports was missed. The extra PyGoat findings are recent OSV records not yet mirrored to PyPI. The 11 extra multer findings are a real gap in npm audit, not over-reporting: Juice Shop uses multer 1.4.5-lts.2, and npm's version matching [10] skips versions with a suffix such as "-lts.2" even when the advisory's range, for example "1.4.4-lts.1 or later, below 2.0.0", plainly covers them. OSV matches them, so sbomrisk ranks multer at #18 with 11 advisories while npm audit would report it clean.

9. Implications for the Final Report

- Report the differentiation target as partly met, with the reasons in Section 3, and present τ and rank changes as the primary evidence of differentiation.
- Report justification and stability as met, including the five unstable runs.
- Present the density result as support for the depth factor, and the multer finding as a practical advantage of OSV-based matching.
- List the Maven release-date gap, upstream over-reporting and data drift as limitations, with their measured effects.

References

[1] Google and OpenSSF, "Open Source Vulnerabilities (OSV) Database and API." [Online]. Available: https://osv.dev

[2] NIST, "National Vulnerability Database API v2.0." [Online]. Available: https://nvd.nist.gov/developers/vulnerabilities

[3] FIRST, "Exploit Prediction Scoring System (EPSS) API." [Online]. Available: https://www.first.org/epss/api

[4] CISA, "Known Exploited Vulnerabilities Catalog." [Online]. Available: https://www.cisa.gov/known-exploited-vulnerabilities-catalog

[5] M. G. Kendall, "A New Measure of Rank Correlation," Biometrika, vol. 30, no. 1/2, pp. 81–93, 1938.

[6] A. Decan, T. Mens, and E. Constantinou, "On the Impact of Security Vulnerabilities in the npm Package Dependency Network," in Proc. 15th Int. Conf. Mining Software Repositories (MSR), 2018, pp. 181–191.

[7] Sonatype, "Maven Central Repository." [Online]. Available: https://central.sonatype.com

[8] Python Packaging Authority, "pip-audit." [Online]. Available: https://github.com/pypa/pip-audit

[9] npm, Inc., "npm audit," npm Docs. [Online]. Available: https://docs.npmjs.com/cli/commands/npm-audit

[10] npm, Inc., "node-semver: The Semantic Versioner for npm." [Online]. Available: https://github.com/npm/node-semver